

INTERVAL TREE-BASED ROOM SCHEDULE CONFLICT DETECTION SYSTEM

I. SYSTEM OVERVIEW

A. System Purpose and Problem Statement

Scheduling classrooms and laboratories is an essential task in colleges and universities. Each academic term, numerous courses must be assigned to available rooms while ensuring that no two classes are scheduled in the same room during overlapping time periods. As the number of subjects, sections, and classrooms increases, managing schedules becomes increasingly complex [4].

In many institutions, schedule conflicts are identified through manual checking, where administrators compare schedules individually to determine whether overlaps exist. While this approach may be manageable for a small number of schedules, it becomes increasingly difficult, time-consuming, and prone to human error as the volume of schedules grows. The process of verifying room availability and detecting conflicts can become inefficient, particularly in colleges where multiple departments share classrooms and laboratory facilities.

The proposed Interval Tree-Based Room Schedule Conflict Detection System aims to provide an efficient method for detecting schedule overlaps using an advanced data structure known as the Interval Tree [5]. The system operates through a console-based interface and focuses primarily on the implementation and analysis of the underlying algorithm rather than graphical interfaces or database integration. By organizing schedules as intervals and utilizing optimized overlap-searching techniques, the system seeks to improve the efficiency of conflict detection compared to traditional linear searching methods. This demonstrates the importance

of automated systems in improving efficiency and organization within higher education institutions[6].

II. ALGORITHM ASSESSMENT

A. Background of the Algorithm

An Interval Tree is a specialized Binary Search Tree (BST) designed for storing intervals and efficiently detecting overlapping intervals [1]. Unlike traditional BSTs that store single values, an Interval Tree stores ranges represented by a starting value and an ending value.

Examples of intervals include:

[8,10]

[11,13]

[14,16]

In the proposed system, each interval represents a room schedule where the start and end values correspond to the beginning and ending times of a class session.

Each node in the Interval Tree contains the following information:

```
class Node {  
  
    int start;  
  
    int end;  
  
    int max;  
  
    String room;  
  
    String subject;  
  
    Node left;
```

```
Node right;  
}
```

The most significant field is the **max** value, which stores the largest ending time among all intervals contained within a node's subtree. During overlap searching, this value allows the algorithm to determine whether a subtree can possibly contain a conflicting interval. If no conflict is possible, the entire subtree can be skipped, reducing the number of comparisons performed.

The overlap condition used in the system is defined as:

$$\text{Overlap} = (\text{start}_1 < \text{end}_2) \ \&\& \ (\text{start}_2 < \text{end}_1) \ (1)$$

For example:

Schedule A: [8,10]

Schedule B: [9,11]

Since:

$$8 < 11$$

$$9 < 10$$

the intervals overlap and therefore represent a scheduling conflict.

The use of subtree maximum values differentiates the Interval Tree from a conventional Binary Search Tree and provides significant performance improvements during overlap detection [1].

B. Time Complexity

Time complexity measures the amount of computational work required by an algorithm relative to the size of the input data.

1) Linear Search The process

In a traditional scheduling system, overlap detection requires checking every stored schedule individually.

Example:

Schedule 1

Schedule 2

Schedule 3

...

Schedule n

The worst-case time complexity is:

$O(n)$

because every schedule may need to be examined.

2) Interval Tree Search

The Interval Tree improves searching efficiency by utilizing subtree maximum values to eliminate branches that cannot contain overlapping intervals.

Instead of checking all nodes, the algorithm traverses only the relevant portions of the tree.

Average-case complexity:

$O(\log n)$

Worst-case complexity:

$O(n)$

The average-case complexity assumes that the Interval Tree remains reasonably balanced. The worst case occurs when the tree becomes highly unbalanced and resembles a linked list, causing search operations to degrade to linear performance.

3) Insertion Operation

Before inserting a schedule, the system first performs conflict detection.

If no conflict exists, the schedule is inserted into the appropriate position within the Interval Tree based on its starting time. The corresponding maximum values are then updated to maintain the integrity of the Interval Tree.

Average-case complexity:

$O(\log n)$

Worst-case complexity:

$O(n)$

As with searching, the average-case complexity assumes a reasonably balanced tree structure.

4) Tree Traversal

The system supports inorder, preorder, and postorder traversals. Since every node must be visited exactly once, traversal complexity is:

$O(n)$

where n represents the number of schedules stored in the tree.

The traversal operations are included not only for displaying schedules but also for demonstrating recursive tree processing and validating the structure of the Interval Tree.

C. Algorithm Mechanics and Logical Flow

The system follows a sequence of operations whenever a user attempts to add a schedule.

Step 1: User Input

The user enters the following information:

- Room Name
- Subject
- Start Time
- End Time

Example Input:

- Room: Lab1
- Subject: DSA
- Start Time: 8
- End Time: 10

The system first validates the schedule to ensure that the start time is less than the end time.

Step 2: Conflict Detection

The system searches the Interval Tree for overlapping schedules within the same room.

The search begins at the root node and recursively traverses the tree. At each node, the algorithm performs the following operations:

1. Compare the target interval with the current interval.
2. Verify whether both schedules belong to the same room.
3. Determine whether an overlap exists using Equation (1).
4. Use the max value to determine whether a subtree should be explored or skipped.

If the maximum ending value stored in the left subtree is less than or equal to the start time of the target interval, the algorithm concludes that the left subtree cannot contain overlapping intervals and safely skips that branch.

This optimization significantly reduces the number of comparisons required during searching.

Step 3: Conflict Found

If an overlap is detected, the system displays:

SCHEDULE CONFLICT DETECTED

The conflicting schedule information is displayed, and the insertion process is terminated.

Example:

SCHEDULE CONFLICT DETECTED

Attempted Schedule:

Room: Lab1

Subject: OOP

Time: 9 - 11

Conflicts With:

Room: Lab1

Subject: DSA

Time: 8 - 10

Step 4: No Conflict Found

If no overlap exists:

1. The schedule is inserted into the tree.
2. The max values of affected nodes are updated.
3. The schedule becomes part of the searchable dataset.

Step 5: Traversal Operations

The system demonstrates three traversal methods:

Inorder Traversal

Left → Root → Right

Displays schedules in ascending order based on their starting times.

Preorder Traversal

Root → Left → Right

Displays the hierarchical structure of the Interval Tree.

Postorder Traversal

Left → Right → Root

Demonstrates recursive tree processing and subtree-first traversal.

D. Comparative Analysis

The primary objective of the project is to compare the performance of Interval Tree searching against traditional linear searching.

Linear Search Approach

Linear search examines every schedule individually without utilizing any subtree information.

Advantages:

- Simple implementation
- Easy to understand

Disadvantages:

- Inefficient for large datasets
- Requires unnecessary comparisons

Complexity:

$O(n)$

Interval Tree Approach

The Interval Tree organizes schedules into a hierarchical structure and utilizes subtree maximum values to reduce unnecessary searching.

Advantages:

- Faster overlap detection
- Reduces unnecessary comparisons
- Demonstrates advanced data structure concepts
- Utilizes subtree pruning through the max field

Disadvantages:

- More complex implementation
- Requires maintenance of max values
- Performance depends on tree balance

Average complexity:

$O(\log n)$

Worst-case complexity:

$O(n)$

To demonstrate the practical performance difference between the two approaches, the system records and displays the number of nodes examined during both Interval Tree searching and Linear Search operations.

TABLE I

PERFORMANCE COMPARISON OF LINEAR SEARCH AND INTERVAL TREE SEARCH

Criteria	Linear Search	Interval Tree
Data Structure	Sequential Checking	Binary Search Tree
Search Strategy	Examine Every Node	Subtree Pruning Using max
Average Search Time	$O(n)$	$O(\log n)$
Worst-Case Search Time	$O(n)$	$O(n)$
Scalability	Limited	Better for Large Datasets
Overlap Detection Efficiency	Low	High
Node Comparisons	Higher	Lower

The comparison demonstrates that the Interval Tree provides a more efficient solution for schedule conflict detection, particularly as the number of schedules increases.

III. DISCUSSION

The Interval Tree-Based Room Schedule Conflict Detection System demonstrates the practical application of advanced data structures in solving real-world scheduling problems. The project highlights how algorithm selection can significantly affect system performance.

By incorporating subtree maximum values, the Interval Tree reduces unnecessary searching and improves the efficiency of overlap detection compared to traditional linear searching methods. The project also reinforces important Data Structures and Algorithms concepts such as Binary Search Trees, recursion, interval searching, tree traversal techniques, and time complexity analysis [1], [2].

Although the proposed system is intentionally limited to a console-based implementation without database integration or graphical interfaces, it successfully fulfills its educational objective of demonstrating the effectiveness of the Interval Tree algorithm. The system provides a clear example of how specialized data structures can optimize search operations and improve computational efficiency.

IV. CONCLUSION

The Interval Tree-Based Room Schedule Conflict Detection System demonstrates how specialized data structures can improve the efficiency of schedule conflict detection. By utilizing subtree maximum values and recursive searching techniques, the Interval Tree reduces unnecessary comparisons and provides better average-case search performance than traditional linear searching methods.

Furthermore, the project successfully illustrates key concepts in Data Structures and Algorithms, including Binary Search Trees, recursion, interval searching, tree traversal operations, and time complexity analysis. As a result, the system serves as both a practical scheduling solution and an effective demonstration of algorithmic optimization.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.
- [3] S. Sahni, *Data Structures, Algorithms, and Applications in Java*, 2nd ed. Hyderabad, India: Universities Press, 2005.
- [4] C. E. B. Pariñas, "Optimizing Academic Scheduling: An Automated Timetabling System for the University of the Philippines Manila," Undergraduate Thesis, University of the Philippines Manila, Manila, Philippines, Jul. 2025.
- [5] X. Hu and J. Y. T. Leung, "Testing Interval Trees for Real-Time Scheduling Systems," in *Proceedings of the 14th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA 2008)*, Kaohsiung, Taiwan, 2008.
- [6] University of the Philippines Mindanao, "Student Academic Information System (SAIS)," University Information System, University of the Philippines Mindanao, Davao City, Philippines. [Online]. Available: [UP Mindanao SAIS](#). [Accessed: Jun. 3, 2026].